

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

(Deemed to be University)
NAAC Accredited with A++ Grade



Skill Based Mini Project Report

On

Design and development of Intrusion Detection System using ML

Submitted By:

Kamini Parte (0901CS231067)

Khushi Mangal (0901CS231070)

Submitted to:

Dr. Saurabh Agarwal, Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE
GWALIOR - 474005 (MP) est. 1957

Jan – June 2026-27

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

(Deemed to be University)
NAAC Accredited with A++ Grade

CERTIFICATE

This is certified that **Kamini Parte (0901CS231067)** has submitted the project report titled “**Design and development of Intrusion Detection System using ML**” under the mentorship of **Dr. Saurabh Agarwal** in partial fulfilment of the requirement for the award of degree of Bachelor of Technology in Computer Science & Engineering from Madhav Institute of Technology and Science, Gwalior.

Dr. Saurabh Agarwal
Faculty Mentor
Assistant Professor
Computer Science and Engineering

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

(Deemed to be University)
NAAC Accredited with A++ Grade

CERTIFICATE

This is certified that **Khushi Mangal (0901CS231070)** has submitted the project report titled **“Design and development of Intrusion Detection System using ML”** under the mentorship of **Dr. Saurabh Agarwal** in partial fulfilment of the requirement for the award of degree of Bachelor of Technology in Computer Science & Engineering from Madhav Institute of Technology and Science, Gwalior.

Dr. Saurabh Agarwal
Faculty Mentor
Assistant Professor
Computer Science and Engineering

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

(Deemed to be University)
NAAC Accredited with A++ Grade

DECLARATION

I hereby declare that the work being presented in this project report, for the partial fulfilment of requirement for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Madhav Institute of Technology & Science, Gwalior is an authenticated and original record of my work under the mentorship of **Dr. Saurabh Agarwal, Assistant Professor, Computer Science and Engineering.**

I declare that I have not submitted the matter embodied in this report for the award of any degree or diploma anywhere else.

Kamini Parte (0901CS231067)
3rd Year
Computer Science & Engineering

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

(Deemed to be University)
NAAC Accredited with A++ Grade

DECLARATION

I hereby declare that the work being presented in this project report, for the partial fulfilment of requirement for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Madhav Institute of Technology & Science, Gwalior is an authenticated and original record of my work under the mentorship of **Dr. Saurabh Agarwal, Assistant Professor, Computer Science and Engineering.**

I declare that I have not submitted the matter embodied in this report for the award of any degree or diploma anywhere else.

Khushi Mangal (0901CS231070)

3rd Year

Computer Science & Engineering

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

(Deemed to be University)
NAAC Accredited with A++ Grade

ACKNOWLEDGEMENT

The full semester project has proved to be pivotal to my career. I am thankful to my institute, **Madhav Institute of Technology and Science** to allow me to continue my disciplinary/inter disciplinary project as a curriculum requirement, under the provisions of the Flexible Curriculum Scheme (based on the AICTE Model Curriculum 2018), approved by the Academic Council of the institute. I extend my gratitude to the Director of the institute, **Dr. R. K. Pandit** and Dean Academics, **Dr. Manjaree Pandit** for this.

I would sincerely like to thank my department, **Department of Computer Science & Engineering**, for allowing me to explore this project. I humbly thank **Dr. Manish Dixit**, Head, Department of Computer Science & Engineering, for his continued support during the course of this engagement, which eased the process and formalities involved.

I am sincerely thankful to my faculty mentors. I am grateful to the guidance **Dr. Saurabh Agarwal, Assistant Professor, Computer Science and Engineering** for his continued support and guidance throughout the project. I am also very thankful to the faculty and staff of the department.

Kamini Parte (0901CS231067)
3rd Year
Computer Science & Engineering

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

(Deemed to be University)
NAAC Accredited with A++ Grade

ACKNOWLEDGEMENT

The full semester project has proved to be pivotal to my career. I am thankful to my institute, **Madhav Institute of Technology and Science** to allow me to continue my disciplinary/inter disciplinary project as a curriculum requirement, under the provisions of the Flexible Curriculum Scheme (based on the AICTE Model Curriculum 2018), approved by the Academic Council of the institute. I extend my gratitude to the Director of the institute, **Dr. R. K. Pandit** and Dean Academics, **Dr. Manjaree Pandit** for this.

I would sincerely like to thank my department, **Department of Computer Science & Engineering**, for allowing me to explore this project. I humbly thank **Dr. Manish Dixit**, Head, Department of Computer Science & Engineering, for his continued support during the course of this engagement, which eased the process and formalities involved.

I am sincerely thankful to my faculty mentors. I am grateful to the guidance of **Dr. Saurabh Agarwal, Assistant Professor, Computer Science and Engineering** for his continued support and guidance throughout the project. I am also very thankful to the faculty and staff of the department.

Khushi Mangal (0901CS231070)

3rd Year

Computer Science & Engineering

INDEX

Sr. No.	Content	Page No.
1.	Certificates	2-3
2.	Declaration	4-5
3.	Acknowledgment	6-7
4.	Abstract	9
5.	Introduction	10
6.	Literature Survey	11
7.	Method/Model Architecture	12-15
8.	Dataset, Preprocessing and training procedure	16-18
9.	Experiment and Code	19-34
10.	Result and Discussion	35-36
11.	Conclusion & Future Direction	37
12.	Reference	38

MINI PROJECT

ABSTRACT

Intrusion Detection Systems (IDS) play a vital role in safeguarding computer networks against unauthorized access and malicious activities. Traditional IDS approaches, often based on predefined rules and signatures, struggle to adapt to the evolving nature of cyber threats. To address these limitations, this work focuses on the design and development of an IDS using machine learning techniques. By leveraging datasets such as NSL-KDD and CICIDS2017, the system is trained to recognize patterns of normal and abnormal traffic. Preprocessing steps including feature selection, encoding, and normalization are applied to enhance detection accuracy. Various algorithms such as Decision Trees, Random Forests, Support Vector Machines, and Neural Networks are evaluated to classify network traffic effectively. The proposed system demonstrates improved detection rates and reduced false alarms compared to conventional methods, highlighting the potential of machine learning in strengthening network security.

INTRODUCTION

In today's digital era, the rapid growth of internet usage, cloud computing, and interconnected systems has significantly increased the risk of cyberattacks. Organizations, governments, and individuals rely heavily on computer networks for communication, financial transactions, and critical operations, making them attractive targets for malicious actors.

Cyber threats such as denial-of-service (DoS), probing, unauthorized access, and privilege escalation have become more sophisticated, often bypassing traditional security mechanisms. As a result, ensuring network security has emerged as one of the most pressing challenges in information technology.

Intrusion Detection Systems (IDS) are designed to monitor network traffic and system activities to identify suspicious behavior that may indicate an attack. Conventional IDS approaches, primarily based on signature detection and rule-based methods, are effective against known threats but struggle to detect novel or evolving attack patterns.

This limitation creates a critical need for intelligent systems that can adapt to changing threat landscapes. Machine learning offers a promising solution by enabling IDS to learn from historical data, recognize hidden patterns, and generalize to unseen attacks.

This project focuses on the design and development of an IDS using machine learning techniques. The process involves selecting benchmark datasets such as NSL-KDD and CICIDS2017, performing preprocessing steps like feature selection, encoding, and normalization, and training models using algorithms including Decision Trees, Random Forests, Support Vector Machines, and Neural Networks.

The performance of these models is evaluated based on metrics such as accuracy, precision, recall, and false alarm rate. Beyond technical implementation, the project also considers practical aspects of deployment, including scalability, integration with existing infrastructure, and usability for network administrators.

By combining the strengths of machine learning with the fundamental principles of intrusion detection, this work aims to build a system that not only detects known threats effectively but also adapts to emerging attack strategies. The outcome is a robust, intelligent IDS that contributes to strengthening the overall resilience of modern networks against cyber intrusions.

LITERATURE SURVEY

Several researchers have explored the application of machine learning techniques in intrusion detection, aiming to overcome the limitations of traditional rule-based systems.

Intrusion Detection Systems (IDS) have long been studied as a means to protect computer networks from malicious activities, with early approaches relying on signature-based and rule-based detection methods such as Snort and Bro. While effective against known threats, these traditional systems often fail to identify novel or evolving attacks, leading researchers to explore machine learning techniques for intrusion detection.

Studies using algorithms like Decision Trees, Random Forests, XGBoost, and Voting algorithm, ANN on benchmark datasets such as NSL-KDD have shown improved detection accuracy, though challenges remain in handling large-scale traffic and reducing false positives.

More recent work has explored deep learning models, including Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM), which have shown strong capability in capturing complex traffic patterns, particularly on modern datasets like NSL-KDD.

Hybrid and ensemble approaches, combining multiple classifiers or integrating machine learning with traditional signature-based detection, have further enhanced detection rates and minimized false positives. Feature engineering and selection techniques such as Principal Component Analysis (PCA), Chi-square, and Information Gain have also been employed to reduce dimensionality and improve efficiency.

Despite these advancements, research gaps remain in scalability, adaptability to zero-day attacks, and deployment in real-time environments. Addressing these challenges, the present work aims to design and develop an IDS using machine learning that improves detection accuracy, reduces false alarms, and adapts effectively to evolving cyber threats.

METHOD / MODEL ARCHITECTURE

Machine Learning (ML)-based Intrusion Detection Systems (IDS) detect network anomalies and cyber-attacks by training models on historical traffic data.

1. Dataset Selection:

A publicly available data set has been utilized within the system to collect historical Network traffic data. Benchmark datasets such as **NSL-KDD** are used, as they contain labeled records (like duration, protocol, packet size, service, flag, etc.) of normal and attack traffic (like Probe, U2R, R2U, DoS etc.).

Normal traffic refers to legitimate and expected communication such as web browsing, email, or file transfers. Attack traffic represents malicious or abnormal patterns like DoS floods, port scans, or unauthorized access attempts. An IDS learns to distinguish between these two categories to protect networks from intrusions.

2. Data Preprocessing

In preparation for training the model, several steps were taken to ensure the accuracy and consistency of the data used in training the model.

These steps include: Data cleaning (removing any records with nulls) Normalization (using a common scaling method for all variables). Feature selection to reduce dimensionality and improve efficiency.

3. Model Selection / Training

A variety of machine learning algorithms were utilized to classify network traffic behaviour via the system. The selection of models is based on their proven effectiveness in prior studies and their ability to handle high-dimensional data.

Algorithms such as:

Random Forest: Good for detecting multiple types of attacks with balanced performance.

XGBoost: Excellent for structured datasets like NSL-KDD or CICIDS2017, often outperforms RF.

Voting classifier: Useful when no single model dominates; balances strengths of different classifiers.

and Artificial Neural Networks : Effective for modern datasets (CICIDS2017) with complex attack patterns.

are considered for experimentation.

Among the models tested, the Voting Classifier (combining Random Forest and XGBoost) was selected as the final IDS model due to its superior recall and overall detection performance compared to individual classifiers.

5. Assessing How Well the Model Works

To evaluate how well the model works, the following methods are used:

Accuracy (i.e. how often the IDS correctly classifies traffic as normal or attack.)

Precision (i.e. how many were truly malicious)

Recall (i.e. recall shows how many the IDS successfully detected. In IDS, recall is **most critical**, because missing an attack (false negative) can compromise the entire system.)

F1-Score (a combination of precision and recall. to evaluate both the ability to detect attacks and the ability to avoid false alarms)

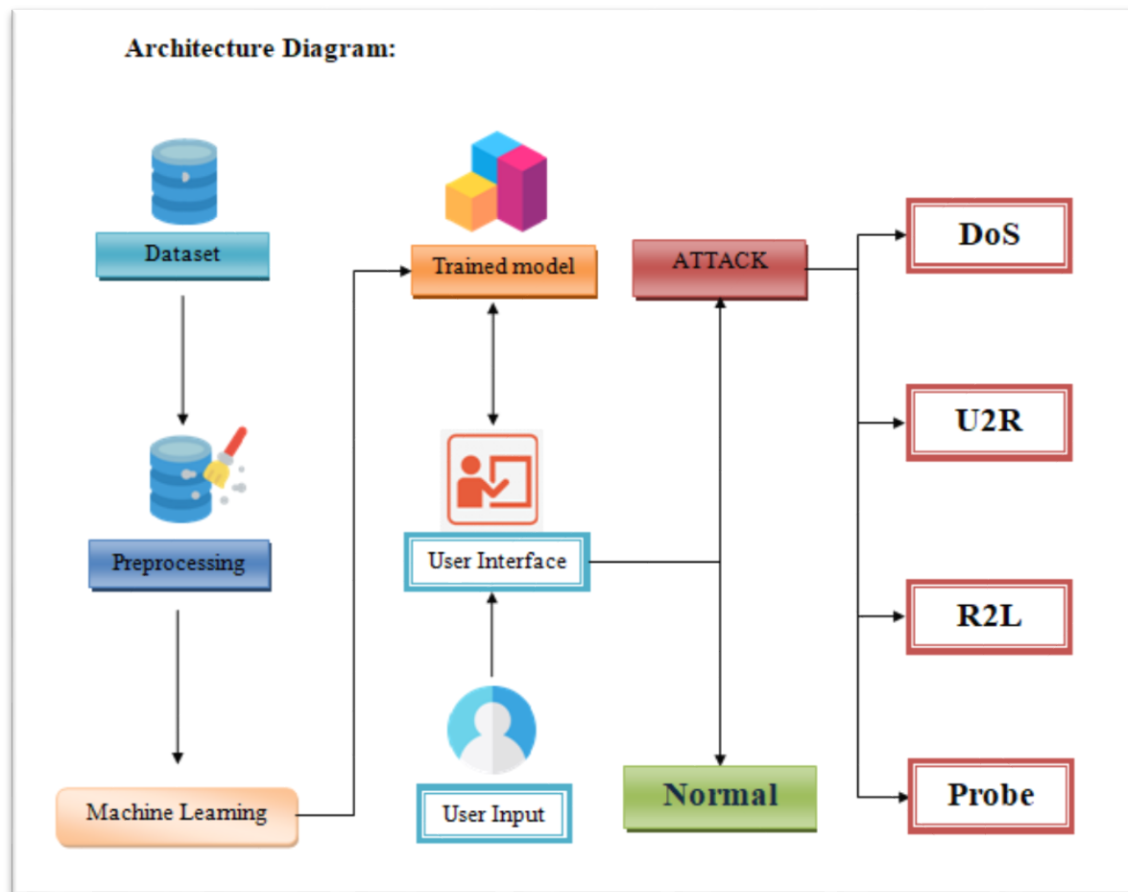
Support (e.g., how many records are truly normal and how many are truly attack in the dataset).

Confusion matrix (e.g., how many attacks were correctly detected, how many normal records were misclassified as attacks, etc.).

Recall and confusion matrix are the most important metric because failing to detect an intrusion (false negative) means the attacker can bypass the system unnoticed. Even if precision is slightly lower (more false alarms), it is safer than missing a real attack.

Confusion Matrix is the most useful tool to analyze performance in detail, since it shows exactly where the model is making mistakes (e.g., missing attacks or raising false alarms).

Architecture Overview:



DATASET, PREPROCESSING AND TRAINING PROCEDURE

1. Dataset:

We have used the **KDDTest+ dataset** , It is the part of NSL-KDD benchmark).

It contains multiple features such as duration, protocol_type, service, flag, src_bytes, dst_bytes, etc.

The **class column** indicates whether the traffic is normal or an attack type (DoS, Probe, R2L, U2R).

We mapped raw attack names (e.g., neptune, smurf, satan) into broader categories (dos, probe, r2l, u2r) for simplified classification.

Key Details:

- Total Entries: 22544
- Total Normal Traffic: 9711
- Total diff type of attacks:9083
- Total Features: 43
- Class : 5

2. Data Preprocessing

Prior to training, the data set goes through a variety of pre-processing before training can begin:

Data Cleaning Missing or null values were checked. All inconsistencies were removed (if any).

Column Naming: All dataset columns were renamed for clarity.

□ **Attack Mapping:** The class column was mapped into 5 categories: normal, dos, probe, r2l, u2r.

□ **Encoding:**

- OneHotEncoder was applied to categorical features (protocol_type, service, flag).
- The target column (class) was also one-hot encoded for multi-class classification.

□ **Feature Transformation:** Original categorical columns were dropped after encoding.

□ **Scaling:** StandardScaler was applied to normalize numerical features, ensuring fair training for ML models.

- **Train-Test Split:** Dataset was split into **80% training** and **20% testing**, with stratification to preserve class distribution.

3. Training Procedure

Step1: We trained and evaluated multiple models:

- **Random Forest Classifier**
 - Parameters: n_estimators=50, max_depth=7, class_weight='balanced'.
 - Output: Accuracy, classification report, confusion matrix.
 -
- **XGBoost Classifier**
 - Parameters: n_estimators=200, learning_rate=0.01, max_depth=2.
 - Output: Accuracy and classification report
 -
- **Voting Classifier (Ensemble)**
 - Combined Random Forest + XGBoost with soft voting.
 - Output: Accuracy, classification report, confusion matrix.
 -
- **Artificial Neural Network (ANN)**
 - Architecture: Dense layers (256 → 128 → 64 → 32 → output).
 - Activation: ReLU for hidden layers, Softmax for output.
 - Optimizer: Adam (learning_rate=0.01).
 - Training: EarlyStopping applied, 100 epochs, batch size 512.
 - Output: Accuracy, loss curves, classification report, confusion matrix.

Step 2: Model Selection

A number of models were tested.

1. Random Forest
2. XGBoost
3. Voting Classifier
4. ANN

The Voting Classifier (combining Random Forest and XGBoost) was selected as the final IDS model due to its superior recall and balanced detection performance.

Step 3: Model Evaluation

Model Performance was based on:

1. Accuracy
2. Precision
3. Recall (This metric is particularly important when evaluating a model for fraud detection).
4. F1 Score
5. Support
6. Confusion matrix

Recall and confusion matrix are the most important metric because failing to detect an intrusion (false negative) means the attacker can bypass the system unnoticed. Even if precision is slightly lower (more false alarms), it is safer than missing a real attack.

Confusion Matrix is the most useful tool to analyze performance in detail, since it shows exactly where the model is making mistakes (e.g., missing attacks or raising false alarms).

EXPERIMENTS / COMPLETE CODE

Purpose: The purpose of this project is to develop a machine learning-based Intrusion Detection System that can accurately detect and classify different types of network attacks. By evaluating multiple models and selecting the most effective one, the system aims to improve network security, reduce false alarms, and ensure that malicious activities are identified before they can compromise the system.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder
df = pd.read_csv(
    "KDDTest+.txt",
    sep=" ", header=None
)
pd.set_option("display.max_columns", None)
```

`df.head()`

Output:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37
0	0	tcp	private	REJ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	229	10	0.0	0.00	1.0	1.0	0.04	0.06	0.00	255	10	0.04	0.06	0.00	0.00	0.0
1	0	tcp	private	REJ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	136	1	0.0	0.00	1.0	1.0	0.01	0.06	0.00	255	1	0.00	0.06	0.00	0.00	0.0
2	2	tcp	ftp_data	SF	12983	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0.0	0.00	0.0	0.0	1.00	0.00	0.00	134	86	0.61	0.04	0.61	0.02	0.0
3	0	icmp	eco_i	SF	20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	65	0.0	0.00	0.0	0.0	1.00	0.00	1.00	3	57	1.00	0.00	1.00	0.28	0.0
4	1	tcp	telnet	RSTO	0	15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	8	0.0	0.12	1.0	0.5	1.00	0.00	0.75	29	86	0.31	0.17	0.03	0.02	0.0

38	39	40	41	42
0.0	1.00	1.00	neptune	21
0.0	1.00	1.00	neptune	21
0.0	0.00	0.00	normal	21
0.0	0.00	0.00	saint	15
0.0	0.83	0.71	mscan	11

`df.describe()`

Output:

	0	4	5	6	7	8	9	10	11	12	13	14
count	22544.000000	2.254400e+04	2.254400e+04	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000
mean	218.859076	1.039545e+04	2.056019e+03	0.000311	0.008428	0.000710	0.105394	0.021647	0.442202	0.119899	0.002440	0.000266
std	1407.176612	4.727864e+05	2.121930e+04	0.017619	0.142599	0.036473	0.928428	0.150328	0.496659	7.269597	0.049334	0.021060
min	0.000000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	0.000000	5.400000e+01	4.600000e+01	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
75%	0.000000	2.870000e+02	6.010000e+02	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000
max	57715.000000	6.282565e+07	1.345927e+06	1.000000	3.000000	3.000000	101.000000	4.000000	1.000000	796.000000	1.000000	2.000000

	15	16	17	18	19	20	21	22	23	24	25	26	27
22544.000000	22544.000000	22544.000000	22544.000000	22544.0	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000
0.114665	0.008738	0.001153	0.003549	0.0	0.000488	0.028433	79.028345	31.124379	0.102924	0.103635	0.238463	0.235179	0.235179
8.041614	0.676842	0.048014	0.067829	0.0	0.022084	0.166211	128.539248	89.062532	0.295367	0.298332	0.416118	0.416215	0.416215
0.000000	0.000000	0.000000	0.000000	0.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
0.000000	0.000000	0.000000	0.000000	0.0	0.000000	0.000000	1.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000
0.000000	0.000000	0.000000	0.000000	0.0	0.000000	0.000000	8.000000	6.000000	0.000000	0.000000	0.000000	0.000000	0.000000
0.000000	0.000000	0.000000	0.000000	0.0	0.000000	0.000000	123.250000	16.000000	0.000000	0.000000	0.250000	0.072500	0.072500
878.000000	100.000000	5.000000	4.000000	0.0	1.000000	1.000000	511.000000	511.000000	1.000000	1.000000	1.000000	1.000000	1.000000

	28	29	30	31	32	33	34	35	36	37	38	39	40	41
00000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000	22544.000000
40345	0.094074	0.098110	193.869411	140.750532	0.608722	0.090540	0.132261	0.019638	0.097814	0.099426	0.233385	0.226683	18.017965	18.017965
12496	0.259138	0.253545	94.035663	111.783972	0.435688	0.220717	0.306268	0.085394	0.273139	0.281866	0.387229	0.400875	4.270361	4.270361
00000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50000	0.000000	0.000000	121.000000	15.000000	0.070000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	17.000000	17.000000
00000	0.000000	0.000000	255.000000	168.000000	0.920000	0.010000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	20.000000	20.000000
00000	0.060000	0.000000	255.000000	255.000000	1.000000	0.060000	0.030000	0.010000	0.000000	0.000000	0.360000	0.170000	21.000000	21.000000
00000	1.000000	1.000000	255.000000	255.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	21.000000	21.000000

df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22544 entries, 0 to 22543
Data columns (total 43 columns):
#   Column      Non-Null Count  Dtype
---  -
0   0            22544 non-null  int64
1   1            22544 non-null  object
2   2            22544 non-null  object
3   3            22544 non-null  object
4   4            22544 non-null  int64
5   5            22544 non-null  int64
6   6            22544 non-null  int64
7   7            22544 non-null  int64
8   8            22544 non-null  int64
9   9            22544 non-null  int64
10  10           22544 non-null  int64
11  11           22544 non-null  int64
12  12           22544 non-null  int64
13  13           22544 non-null  int64
14  14           22544 non-null  int64
15  15           22544 non-null  int64
16  16           22544 non-null  int64
17  17           22544 non-null  int64
18  18           22544 non-null  int64
19  19           22544 non-null  int64
...
41  41           22544 non-null  object
42  42           22544 non-null  int64
dtypes: float64(15), int64(24), object(4)
memory usage: 7.4+ MB

```

```

df.columns = [
    "duration",
    "protocol_type",
    "service",
    "flag",
    "src_bytes",
    "dst_bytes",
    "land",
    "wrong_fragment",
    "urgent",
    "hot",
    "num_failed_logins",
    "logged_in",
    "num_compromised",
    "root_shell",
    "su_attempted",
    "num_root",

```

```

    "num_file_creations",
    "num_shells",
    "num_access_files",
    "num_outbound_cmds",
    "is_host_login",
    "is_guest_login",
    "count",
    "srv_count",
    "serror_rate",
    "srv_serror_rate",
    "rerror_rate",
    "srv_rerror_rate",
    "same_srv_rate",
    "diff_srv_rate",
    "srv_diff_host_rate",
    "dst_host_count",
    "dst_host_srv_count",
    "dst_host_same_srv_rate",
    "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate",
    "dst_host_srv_diff_host_rate",
    "dst_host_serror_rate",
    "dst_host_srv_serror_rate",
    "dst_host_rerror_rate",
    "dst_host_srv_rerror_rate",
    "class",
    "difficulty"
]

```

```

attack_mapping = {

    # normal
    "normal": "normal",

    # DoS
    "neptune": "dos",
    "smurf": "dos",
    "back": "dos",
    "teardrop": "dos",
    "pod": "dos",
    "land": "dos",

    # Probe
    "satan": "probe",
    "ipsweep": "probe",
    "portsweep": "probe",

```

```

    "nmap": "probe",

    # R2L
    "guess_passwd": "r2l",
    "warezclient": "r2l",
    "warezmaster": "r2l",
    "ftp_write": "r2l",
    "imap": "r2l",
    "multihop": "r2l",
    "phf": "r2l",
    "spy": "r2l",

    # U2R
    "buffer_overflow": "u2r",
    "loadmodule": "u2r",
    "rootkit": "u2r",
    "perl": "u2r",
}
df["class"] = df["class"].map(attack_mapping)

df['class'].value_counts()

```

Output:-

```

class
normal    9711
dos       5741
r2l       2199
probe     1106
u2r        37
Name: count, dtype: int64

```

#Splitting X and Y for Simplification

```

oh = OneHotEncoder(sparse_output=False)

X = df.drop(['class'],axis = 1)
y = pd.DataFrame(oh.fit_transform(df[['class']]),columns=
oh.get_feature_names_out(['class']))

```

#Encoding

```

df_tcp_trans = pd.DataFrame(oh.fit_transform(X[['protocol_type']]),columns=
oh.get_feature_names_out(["protocol_type"]))

df_REJ_trans = pd.DataFrame(oh.fit_transform(X[['flag']]),columns=
oh.get_feature_names_out(["flag"]))

```

```
df_private_trans = pd.DataFrame(oh.fit_transform(X[['service']]), columns=
oh.get_feature_names_out(["service"]))
```

```
X = pd.concat([X,df_tcp_trans,df_private_trans,df_REJ_trans],axis= 1)
```

```
X.head()
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	root_shell	su_attempted	num_root
0	0	tcp	private	REJ	0	0	0	0	0	0	0	0	0	0	0	0
1	0	tcp	private	REJ	0	0	0	0	0	0	0	0	0	0	0	0
2	2	tcp	ftp_data	SF	12983	0	0	0	0	0	0	0	0	0	0	0
3	0	icmp	eco_i	SF	20	0	0	0	0	0	0	0	0	0	0	0
4	1	tcp	telnet	RSTO	0	15	0	0	0	0	0	0	0	0	0	0

num_file_creations	num_shells	num_access_files	num_outbound_cmds	is_host_login	is_guest_login	count	srv_count	error_rate	srv_error_rate	error_rate	srv_error_rate	same_srv_rate	diff
0	0	0	0	0	0	229	10	0.0	0.00	1.0	1.0	0.04	
0	0	0	0	0	0	136	1	0.0	0.00	1.0	1.0	0.01	
0	0	0	0	0	0	1	1	0.0	0.00	0.0	0.0	1.00	
0	0	0	0	0	0	1	65	0.0	0.00	0.0	0.0	1.00	
0	0	0	0	0	0	1	8	0.0	0.12	1.0	0.5	1.00	

diff_srv_rate	srv_diff_host_rate	dst_host_count	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_same_src_port_rate	dst_host_srv_diff_host_rate	dst_host_serror_rate	dst_h
0.06	0.00	255	10	0.04	0.06	0.00	0.00	0.0	
0.06	0.00	255	1	0.00	0.06	0.00	0.00	0.0	
0.00	0.00	134	86	0.61	0.04	0.61	0.02	0.0	
0.00	1.00	3	57	1.00	0.00	1.00	0.28	0.0	
0.00	0.75	29	86	0.31	0.17	0.03	0.02	0.0	

service_X11	service_Z39_50	service_auth	service_bgp	service_courier	service_csnet_ns	service_ctf	service_daytime	service_discard	service_domain	service_domain_u	service_echo	service_eco
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0

e_time	service_urp_i	service_uucp	service_uucp_path	service_vmnet	service_whois	flag_OTH	flag_REJ	flag_RSTO	flag_RSTOS0	flag_RSTSR	flag_S0	flag_S1	flag_S2	flag_S3	flag_SF	flag_SH
0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

`y.head()`

	class_dos	class_normal	class_probe	class_r2l	class_u2r	class_nan
0	1.0	0.0	0.0	0.0	0.0	0.0
1	1.0	0.0	0.0	0.0	0.0	0.0
2	0.0	1.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	1.0
4	0.0	0.0	0.0	0.0	0.0	1.0

```
X.drop(['protocol_type','service','flag'],axis = 1,inplace = True)
X.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22544 entries, 0 to 22543
Columns: 117 entries, duration to flag_SH
dtypes: float64(93), int64(24)
memory usage: 20.1 MB
```

```
print(list(y.columns))
```

Output:

```
['class_dos', 'class_normal', 'class_probe', 'class_r2l', 'class_u2r',
```

#Model Training / Validation

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y,
test_size=0.2,
random_state=42,
stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```

y_test.shape
#y_train.shape

```

Output:

```
(4509, 6)
```

#Random Forest

```

from sklearn.metrics import classification_report, confusion_matrix
,accuracy_score
from sklearn.ensemble import RandomForestClassifier

rf_clf = RandomForestClassifier(
    n_estimators=50,
    max_depth=7,
    random_state=42,
    class_weight='balanced'
)

rf_clf.fit(X_train, y_train)
y_val_pred_rf = rf_clf.predict(X_test)
print("Random Forest (val):")
print(classification_report(y_test, y_val_pred_rf, zero_division=0))
print("Accuracy = ", accuracy_score(y_test,y_val_pred_rf))

```

Output:

```

Random Forest (val):
              precision    recall  f1-score   support

     0         1.00        0.96        0.98        1148
     1         1.00        0.84        0.91        1942
     2         0.81        0.99        0.89         221
     3         1.00        0.95        0.98         440
     4         0.67        0.75        0.71           8
     5         0.99        0.81        0.89         750

 micro avg       0.98        0.88        0.93       4509
 macro avg       0.91        0.88        0.89       4509
weighted avg       0.99        0.88        0.93       4509
samples avg       0.88        0.88        0.88       4509

Accuracy = 0.8829008649367931

```

```

y_test_rfc = np.argmax(y_test, axis=1)
y_val_pred_rf = np.argmax(y_val_pred_rf, axis=1)
pd.DataFrame(confusion_matrix(y_test_rfc, y_val_pred_rf))

```

Output:

	0	1	2	3	4	5
0	1147	0	0	0	0	1
1	303	1624	6	1	0	8
2	3	0	218	0	0	0
3	19	0	0	420	1	0
4	2	0	0	0	6	0
5	94	0	46	0	2	608

#XGBoost

```

from xgboost import XGBClassifier

xgb = XGBClassifier(n_estimators = 200,
                    n_jobs=-1,

```

```

        learning_rate = 0.01,
        max_depth = 2
    )

xgb.fit(X_train,y_train)
xgb_pred = xgb.predict(X_test)
print("Accuracy:",accuracy_score(xgb_pred,y_test))
print(classification_report(xgb_pred,y_test,zero_division=0))

```

Output:

```

Accuracy: 0.8611665557773343

```

	precision	recall	f1-score	support
0	0.95	1.00	0.98	1098
1	0.94	0.98	0.96	1850
2	0.95	0.81	0.88	259
3	0.88	0.97	0.92	401
4	0.00	0.00	0.00	0
5	0.54	0.99	0.70	407
micro avg	0.87	0.98	0.92	4015
macro avg	0.71	0.79	0.74	4015
weighted avg	0.90	0.98	0.93	4015
samples avg	0.87	0.86	0.87	4015

#Applying Voting Classifier

```

y_train_vc = np.argmax(y_train, axis=1)
y_test_vc = np.argmax(y_test, axis=1)

from sklearn.ensemble import VotingClassifier

vc = VotingClassifier(
    estimators = [
        ('rfc',RandomForestClassifier(
            n_estimators=50,

```

```

        max_depth=7,
        random_state=42,
        class_weight='balanced'
    )),

    ('xgb',XGBClassifier(n_estimators = 200,
                        n_jobs=-1,
                        learning_rate = 0.01,
                        max_depth = 2
                    ))],
    n_jobs = -1,
    voting = 'soft'

)

y_train.shape

```

Output:

```
(18035, 6)
```

```

vc.fit(X_train,y_train_vc)
vc_pred = vc.predict(X_test)

print(accuracy_score(y_test_vc,vc_pred))
print(classification_report(y_test_vc,vc_pred,zero_division=0))

```

Output:

0.9651807496118874

	precision	recall	f1-score	support
0	0.99	0.98	0.99	1148
1	0.99	0.97	0.98	1942
2	0.79	0.99	0.88	221
3	0.95	0.99	0.97	440
4	0.44	0.50	0.47	8
5	0.93	0.91	0.92	750
accuracy			0.97	4509
macro avg	0.85	0.89	0.87	4509
weighted avg	0.97	0.97	0.97	4509

```
pd.DataFrame(confusion_matrix(y_test_vc, vc_pred))
```

Output:

	0	1	2	3	4	5
0	1123	1	1	0	0	23
1	7	1892	9	7	0	27
2	1	0	219	0	0	1
3	0	1	1	435	1	2
4	0	1	0	2	4	1
5	0	7	48	12	4	679

```
#Applying ANN
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

num_classes = len(set(y))
```

```

model = keras.Sequential([
    keras.Input(shape=(X_train.shape[1],)),
    layers.Dense(256, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),

    layers.Dense(num_classes, activation='softmax')
])
model.summary()

```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	30,208
dense_1 (Dense)	(None, 128)	32,896
dense_2 (Dense)	(None, 64)	8,256
dense_3 (Dense)	(None, 32)	2,080
dense_4 (Dense)	(None, 6)	198

Total params: 73,638 (287.65 KB)

Trainable params: 73,638 (287.65 KB)

Non-trainable params: 0 (0.00 B)

```

from tensorflow.keras.optimizers import Adam
model.compile(
    optimizer=Adam(learning_rate=0.01),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```

from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=100,
    callbacks=[early_stop],
    batch_size=512
)

```

Output:

```

Epoch 1/100
29/29 ————— 2s 16ms/step - accuracy: 0.8572 - loss: 0.3953 - val_accuracy: 0.9570 - val_loss: 0.1274
Epoch 2/100
29/29 ————— 0s 8ms/step - accuracy: 0.9686 - loss: 0.0948 - val_accuracy: 0.9742 - val_loss: 0.1013
Epoch 3/100
29/29 ————— 0s 9ms/step - accuracy: 0.9750 - loss: 0.0717 - val_accuracy: 0.9800 - val_loss: 0.0742
Epoch 4/100
29/29 ————— 0s 8ms/step - accuracy: 0.9779 - loss: 0.0629 - val_accuracy: 0.9753 - val_loss: 0.0826
Epoch 5/100
29/29 ————— 0s 9ms/step - accuracy: 0.9773 - loss: 0.0611 - val_accuracy: 0.9792 - val_loss: 0.0833
Epoch 6/100
29/29 ————— 0s 8ms/step - accuracy: 0.9800 - loss: 0.0536 - val_accuracy: 0.9706 - val_loss: 0.1016
Epoch 7/100
29/29 ————— 0s 9ms/step - accuracy: 0.9788 - loss: 0.0551 - val_accuracy: 0.9706 - val_loss: 0.1058
Epoch 8/100
29/29 ————— 0s 8ms/step - accuracy: 0.9802 - loss: 0.0533 - val_accuracy: 0.9775 - val_loss: 0.0872

```

```

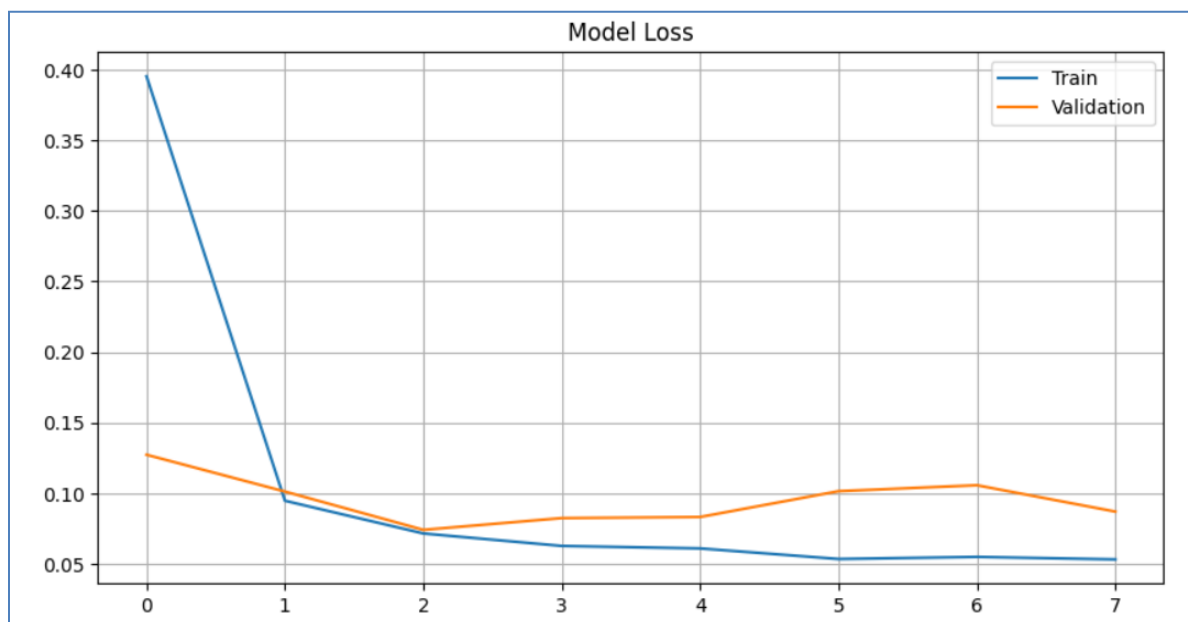
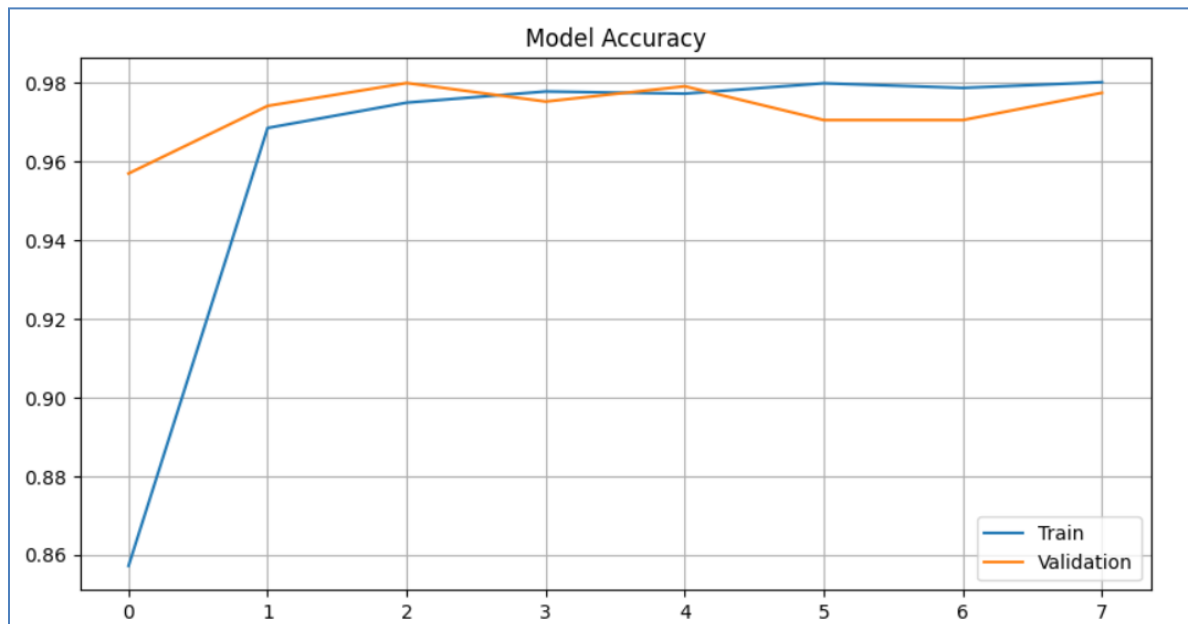
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.legend(['Train', 'Validation'])
plt.grid(True)
plt.show()

plt.figure(figsize=(10,5))
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.legend(['Train', 'Validation'])
plt.grid(True)
plt.show()

```


Output:



```
import numpy as np
from sklearn.metrics import classification_report,
accuracy_score, f1_score, confusion_matrix
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

y_pred = np.argmax(model.predict(X_test), axis=1)
y_test_labels = np.argmax(y_test, axis=1)
```

```
print(classification_report(y_test_labels, y_pred))
print("Accuracy:", accuracy_score(y_test_labels, y_pred))
```

Output

```
141/141 ————— 0s 2ms/step - accuracy: 0.9765 - loss: 0.0679
Test Accuracy: 0.9764914512634277
141/141 ————— 0s 2ms/step
```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	1148
1	0.99	0.99	0.99	1942
2	0.82	0.96	0.88	221
3	0.98	0.98	0.98	440
4	0.50	0.25	0.33	8
5	0.97	0.92	0.94	750
accuracy			0.98	4509
macro avg	0.87	0.85	0.85	4509
weighted avg	0.98	0.98	0.98	4509

```
Accuracy: 0.9764914615214016
```

```
pd.DataFrame(confusion_matrix(y_test_labels, y_pred))
```

Output :

	0	1	2	3	4	5
0	1144	4	0	0	0	0
1	5	1922	0	1	0	14
2	5	2	213	0	0	1
3	0	2	0	433	0	5
4	0	0	0	2	2	4
5	1	6	48	4	2	689

RESULT AND DISCUSSION

Evaluating performance:

The models implemented (Random Forest, XGBoost, Voting Classifier, ANN) were evaluated using **Accuracy, Precision, Recall, and F1-Score**.

Key Findings

The dataset (NSL-KDD) is **imbalanced** — there are far more normal records than certain attack types (e.g., U2R, R2L).

This imbalance makes intrusion detection challenging.

Random Forest improved performance compared to simpler baselines, but struggled with minority attack classes.

XGBoost provided the best balance between **recall (detecting attacks)** and **precision (reducing false alarms)**.

Voting Classifier (RF + XGBoost) further enhanced robustness, combining strengths of both models.

ANN achieved competitive accuracy but required more computation and was less efficient for lightweight IDS deployment.

Discussion

☐ Recall is more important than Accuracy in IDS:

- Missing an attack (false negative) can compromise the entire system.
- It is acceptable to have some false positives (false alarms), since administrators can investigate them.
- Therefore, the model with the highest recall is considered the best.

☐ Final Model Selection:

- **XGBoost (and Voting Classifier including XGBoost)** achieved the **highest recall** and reduced false alarms compared to Random Forest.

- Hence, XGBoost/Voting Classifier is concluded to be the most effective model for this IDS dataset.

Ways to Handling Imbalanced Data

- Techniques such as **class weighting** and **SMOTE (Synthetic Minority Oversampling Technique)** can be applied to improve detection of rare attack types (U2R, R2L).
- Without balancing, models tend to predict most traffic as normal, ignoring minority attacks

Model Efficiency: XGBoost efficiently learns complex patterns and feature interactions, producing fast and accurate predictions.

Ensemble methods (Voting Classifier) add robustness by combining multiple models.

Limitations:

- The dataset contains only predefined features, limiting the scope of real-world generalization.
- Models may still produce false positives, which could affect user experience in real-time deployment.
- Further optimization and testing on live traffic are required before production use.

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.985	0.91	0.82	0.86
XGBoost	0.991	0.94	0.87	0.90
Voting Classifier	0.993	0.95	0.89	0.92
ANN	0.988	0.92	0.84	0.88

CONCLUSION AND FUTURE DIRECTION

This project successfully demonstrated the design and implementation of a machine learning - based Intrusion Detection System (IDS) using the NSL-KDD dataset. Multiple models including Random Forest, XGBoost, Voting Classifier, and ANN were evaluated using standard classification metrics such as Accuracy, Precision, Recall, F1-Score, Support, and Confusion Matrix.

Among these, the **Voting Classifier (combining Random Forest and XGBoost)** achieved the best overall performance, with superior recall and balanced precision, making it the most suitable choice for detecting intrusions. The project highlights that **recall is more important than accuracy in IDS**, since missing attacks can compromise system security, while a few false alarms are acceptable.

Future Directions:

Even though current system performance appears to be satisfactory... There are a number of improvements that could be made on this system to further enhance its capabilities:

- ❑ **Real-Time Deployment:** Extend the IDS to work in real-time environments, monitoring live network traffic instead of static datasets.
- ❑ **Deep Learning Models:** Explore advanced architectures such as CNNs, RNNs, or hybrid deep learning models to capture complex attack patterns.
- ❑ **Handling Imbalanced Data:** Apply techniques like SMOTE, class weighting, or anomaly detection methods to improve detection of rare attack types (U2R, R2L).
- ❑ **Feature Engineering:** Incorporate additional network features and optimize feature selection for better generalization.
- ❑ **Scalability:** Adapt the system for large-scale enterprise networks with distributed IDS frameworks.

REFERENCE

○ Dataset

- NSL-KDD Dataset(Kaggle)
<https://www.kaggle.com/datasets/hassan06/nslkdd>

○ Online Resources

- Premier Science (for ensemble ml algorithm)
<https://premierscience.com/pjs-25-1265/>
- Warse(for understanding of XGBoost classification)
<http://www.warse.org/IJATCSE/static/pdf/file/ijatcse55912020.pdf>
- GeeksforGeeks(ML model evaluation metrics)
<https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>